

MODULE

5

Record Functions

Master every Record.* function in M — from field access and inspection through creation, modification, dynamic transformation, and metadata operations. Build row-level logic, config-driven queries, and inline lookups that make you a professional-grade Power Query developer.

Module 5: Record Functions

Module Contents

■ Module Overview

■ Lesson 1: Understanding the Record Type

- What Is a Record in M?
- Records vs Lists vs Tables
- Field Access: `[]` Syntax vs `Record.Field()`
- Type Annotations and Nested Records

■ Lesson 2: Accessing and Inspecting Records

- `Record.Field` — Runtime-Dynamic Field Access
- `Record.FieldNames` — List of Field Names
- `Record.FieldValues` — List of Field Values
- `Record.FieldCount` and `Record.HasFields`
- `Record.FieldOrDefault` — Safe Access with Fallback
- `Record.ToTable` and `Record.ToList`

■ Lesson 3: Creating and Modifying Records

- Record Literals: `[]` Syntax
- `Record.AddField` — Adding Fields
- `Record.RemoveFields` — Removing Fields
- `Record.RenameFields` — Renaming Fields
- `Record.SelectFields` — Keeping Only Specified Fields

■ Lesson 4: Transforming and Combining Records

- `Record.TransformFields` — Modify Existing Field Values
- `Record.Combine` — Merge Multiple Records
- `Record.FromList` and `Record.FromTable`

■ Lesson 5: Records in Tables — Row-Level Patterns

- `each _` Is a Record
- Multi-Column Computed Record Pattern
- Inline Lookup Without `Table.Join`

■ Lesson 6: Metadata and Type Operations

- `Value.Metadata` and `Value.ReplaceMetadata`
- Annotating Types with Metadata Records
- Dynamic Type Builder for `Table.TransformColumnTypes`

■ Lesson 7: Advanced Record Patterns for PL-300

- Pattern 1: Config-Driven Query with `Record.FromTable`
- Pattern 2: Row Comparison (Previous Row Delta)
- Pattern 3: Flattening Nested JSON Records
- Pattern 4: Parameterised Column Selector
- Pattern 5: Defaults-and-Overrides Function Options

■ Demonstration 1: Config Record from an Excel Parameter Table

■ Demonstration 2: Flattening a Nested JSON API Response

■ Lab 5: Record Functions

- Exercise 1: Record Access and Inspection

- Exercise 2: Creating and Modifying Records
- Exercise 3: Records in Tables — Row-Level Patterns
- Exercise 4: Config Records and Dynamic Type Schemas

■ **Module Review and Takeaways**

- Review Questions
- Best Practices
- Common Issues and Solutions
- Key Functions Quick Reference

Module Overview

The record is the most intimate data structure in M. While most Power Query developers encounter records passively — every table row is a record, every `each` expression operates on a record — very few understand them deeply enough to exploit their full potential. A developer who masters records can build config-driven queries that require zero code changes to adapt to new environments, perform inline lookups without expensive joins, flatten nested JSON from REST APIs without manual steps, and construct dynamic type schemas that self-document the data model.

This module covers the complete `Record.*` function library — from the fundamentals of creation and field access through modification, dynamic transformation, metadata annotation, and five professional-grade patterns directly applicable to PL-300 exam scenarios and production Power BI models. Each lesson builds on the previous, culminating in two instructor demonstrations and a four-exercise lab.

NOTE

This module assumes familiarity with Modules 1–4. You should be comfortable with the `let...in` structure, `Table.AddColumn`, `Table.SelectRows`, the `each` keyword, list functions, and type annotations before proceeding.

Before You Begin

- Complete Modules 1–4 and their lab exercises.
- Have the `FactSales`, `DimProduct`, and `DimDate` queries from previous labs loaded in Power BI Desktop.
- Download the lab support file: `Config_Parameters.xlsx` from the companion materials (simulates a SharePoint configuration table).

LESSON 1

Understanding the Record Type

After completing this lesson, you will be able to:

- Describe what a record is in M and how it differs from a list and a table.
- Write record literals using `[]` syntax and access fields with `[]` and `Record.Field()`.
- Explain why `Record.Field()` is required for runtime-dynamic field access.
- Recognise that every table row is a record and every `each _` expression operates on one.
- Apply type annotations to record values.

What Is a Record in M?

A record in M is an ordered collection of named fields, where each field has a name and a value of any type. Records are written between square brackets:

```
// A simple record literal
Customer = [ Name = "Alice", Age = 30, Score = 95.5 ]

// Fields can hold any M value – lists, other records, dates, nulls
Order = [
  OrderID = "CA-001",
  Lines = {1200, 850, 325}, // a list
  Address = [ City = "Seattle", Zip = "98101" ], // nested record
  Placed = #date(2024, 3, 15),
  Shipped = null
]

// An empty record
Empty = []
```

Records vs Lists vs Tables

Understanding how these three structured types relate is fundamental to advanced M. Every table row is a record; every table column extracted with `table[Col]` is a list:

Type	Structure	Access Pattern	Analogy
Record	Named field-value pairs	<code>[FieldName]</code> or <code>Record.Field()</code>	A single row of a table
List	Ordered sequence of values	<code>{n}</code> index	A column extracted from a table
Table	Named, typed columns with multiple rows	<code>[Col]</code> or <code>{row}</code>	A spreadsheet / database table

```
// Given a table called Sales:
// | OrderID | Amount | Region |
// | CA-001 | 1200   | West   |

// Get the Amount column as a list:
AmountList = Sales[Amount] // = {1200, ...}

// Get row 0 as a record:
FirstRow = Sales{0} // = [OrderID="CA-001", Amount=1200, Region="West"]
```

```
// Get a specific cell:  
FirstAmt = Sales{0}[Amount] // = 1200
```

Field Access: [] Syntax vs Record.Field()

There are two ways to read a record field. The choice between them is not style — it determines whether dynamic (runtime) access is possible:

```
rec = [ Name = "Alice", Age = 30 ]  
  
// Compile-time syntax: field name must be a literal string  
rec[Name] // = "Alice" ✓ – always works  
rec[Age] // = 30 ✓  
  
// Runtime-dynamic: field name can be a variable  
fieldName = "Age"  
Record.Field(rec, fieldName) // = 30 ✓ – only this form works  
rec[fieldName] // ERROR – [] requires a literal, not a variable
```

NOTE

Use `rec[Field]` for interactive queries where field names are known at design time. Use `Record.Field(rec, varName)` whenever the field name is computed at runtime — this is mandatory for dynamic column patterns, config-driven queries, and generic transformation functions.

LESSON 2

Accessing and Inspecting Records

After completing this lesson, you will be able to:

- Access record fields dynamically using `Record.Field`.
- List all field names and values with `Record.FieldNames` and `Record.FieldValues`.
- Count fields and test for field presence with `Record.FieldCount` and `Record.HasFields`.
- Read a field safely with a fallback using `Record.FieldOrDefault`.
- Convert a record to a table or list for further processing.

Syntax	<code>Record.Field(record as record, field as text) as any</code>	
Parameter	Type	Description
<code>record</code>	record	The source record.
<code>field</code>	text	The name of the field to retrieve. May be a variable — this is the key difference from the <code>[]</code> operator.
Returns	Returns the value of the named field. Throws an error if the field does not exist.	
Example	<code>Record.Field([Name="Alice", Age=30], "Age")</code>	
Result	30	

NOTE

The most important use of `Record.Field` over `rec[Field]` is when the field name is stored in a variable: `Record.Field(rec, myField)` works; `rec[myField]` does not. Use this in every generic function that processes unknown schemas.

Syntax	<code>Record.FieldNames(record as record) as list</code>	
Parameter	Type	Description
<code>record</code>	record	The source record.
Returns	Returns a list of all field names in the record, in field order.	
Example	<code>Record.FieldNames([X=10, Y=20, Z=30])</code>	
Result	{ "X", "Y", "Z" }	

NOTE

Pair with `Table.ColumnNames` to compare record structure against a table schema. Also used in `Record.TransformFields` loops where you do not know field names in advance.

Syntax	Record.FieldValues(record as record) as list	
Parameter	Type	Description
record	record	The source record.
Returns	Returns a list of all field values in the record, in the same order as Record.FieldNames.	
Example	Record.FieldValues([X=10, Y=20])	
Result	{10, 20}	

NOTE

Pair with Record.FieldNames to decompose a record into parallel key/value lists. Use List.Zip({Record.FieldNames(r), Record.FieldValues(r)}) to get {name, value} pairs.

Syntax	Record.FieldCount(record as record) as number	
Parameter	Type	Description
record	record	The source record.
Returns	Returns the number of fields in the record.	
Example	Record.FieldCount([A=1, B=2, C=3])	
Result	3	

NOTE

Equivalent to List.Count(Record.FieldNames(record)). Useful for validating that a record has the expected number of fields before processing.

Syntax	Record.HasFields(record as record, fields as any) as logical	
Parameter	Type	Description
record	record	The source record.
fields	text or list	A single field name (text) or a list of field names. Returns true only if ALL specified fields are present.
Returns	Returns true if the record contains all specified fields; false otherwise.	
Example	Record.HasFields([A=1, B=2], {"A", "C"})	
Result	false	

NOTE

Use at the start of any transformation that requires specific fields to be present. Pair with an error expression: if not Record.HasFields(row, RequiredFields) then error "Missing columns" else ... This prevents cryptic downstream failures.

Syntax	<code>Record.FieldOrDefault(record as record, field as text, optional defaultValue as any) as any</code>	
Parameter	Type	Description
<code>record</code>	record	The source record.
<code>field</code>	text	The name of the field to retrieve.
<code>defaultValue</code>	any	Optional. Returned if the field does not exist. Default: null.
Returns	Returns the value of the named field, or <code>defaultValue</code> if the field is not present. Never throws an error for missing fields.	
Example	<code>Record.FieldOrDefault([A=1, B=2], "C", 0)</code>	
Result	0	

NOTE

This is the safe version of `Record.Field`. Always use it for lookup operations where unmatched keys are possible — for example, `Record.FieldOrDefault(RegionLookup, [RegionCode], "Unknown")`. `Record.Field` would throw; `Record.FieldOrDefault` returns your default.

Syntax	<code>Record.ToTable(record as record) as table</code>	
Parameter	Type	Description
<code>record</code>	record	The source record.
Returns	Converts the record to a two-column table with columns named <code>Name</code> (text) and <code>Value</code> (any). Each field becomes one row.	
Example	<code>Record.ToTable([City="Tokyo", Pop=14000000])</code>	
Result	table: Name="City" Value="Tokyo" Name="Pop" Value=14000000	

NOTE

The inverse operation is `Record.FromTable(table)`. This pair is the foundation of config-driven queries: load a `Name/Value` table from SharePoint or Excel, convert to a record with `Record.FromTable`, then access parameters by name.

LESSON 3

Creating and Modifying Records

After completing this lesson, you will be able to:

- Create records using `[]` literal syntax with any field value types.
- Add new fields to a record with `Record.AddField`.
- Remove one or more fields with `Record.RemoveFields`.
- Rename fields with `Record.RenameFields`.
- Keep only specified fields with `Record.SelectFields`.

Records in M are **immutable** — no function modifies a record in place. Every modification function returns a new record with the change applied. This functional approach makes transformations composable and free of side effects.

Syntax	Record.AddField(record as record, fieldName as text, value as any, optional delayed as nullable logical) as record	
Parameter	Type	Description
record	record	The source record.
fieldName	text	The name of the new field to add.
value	any	The value for the new field.
delayed	logical	Optional. If true, value is a zero-argument function evaluated lazily. Used to resolve circular references.
Returns	Returns a new record with the specified field appended. The original record is unchanged.	
Example	Record.AddField([A=1, B=2], "C", 3)	
Result	[A=1, B=2, C=3]	

NOTE

delayed=true is rarely needed but important for self-referential records. Example:
 Record.AddField([A=1], "B", () => [A]*2, true) defers the evaluation of B until it is first accessed.

Syntax	Record.RemoveFields(record as record, fields as any) as record	
Parameter	Type	Description
record	record	The source record.
fields	text or list	A single field name (text) or a list of field names to remove.
Returns	Returns a new record with the specified field(s) removed. Throws if a named field does not exist.	
Example	Record.RemoveFields([A=1, B=2, C=3], "B")	
Result	[A=1, C=3]	

NOTE

To safely remove fields that may not exist, intersect with Record.FieldNames first: ToRemove = List.Intersect({FieldsToRemove, Record.FieldNames(rec)}). This prevents errors when the source schema varies between runs.

Syntax	Record.RenameFields(record as record, renames as list) as record	
Parameter	Type	Description
record	record	The source record.
renames	list	A list of {oldName, newName} pairs. Example: {"OldA", "NewA"}, {"OldB", "NewB"}.
Returns	Returns a new record with the specified fields renamed. Field order is preserved.	
Example	Record.RenameFields([Nm="Bob", Ag=25], {"Nm", "Name"}, {"Ag", "Age"})	
Result	[Name="Bob", Age=25]	

NOTE

For dynamic renames from a config table, build the pairs with List.Zip: Pairs = List.Zip({OldNameList, NewNameList}). This lets you maintain rename mappings in Excel or SharePoint without changing M code.

Syntax	Record.SelectFields(record as record, fields as any, optional missingField as nullable number) as record	
Parameter	Type	Description
record	record	The source record.
fields	text or list	A field name or list of field names to keep.
missingField	number	Optional. MissingField.Ignore or MissingField.Error.
Returns	Returns a new record containing only the specified fields, in the specified order.	
Example	Record.SelectFields([A=1, B=2, C=3, D=4], {"A","C"})	
Result	[A=1, C=3]	

NOTE

This is the record equivalent of Table.SelectColumns. Use it inside Table.AddColumn to extract a sub-record per row before expanding it into new columns.

LESSON 4

Transforming and Combining Records

After completing this lesson, you will be able to:

- Apply transformation functions to specific record fields with `Record.TransformFields`.
- Merge multiple records into one with `Record.Combine`, understanding the last-wins rule.
- Build records programmatically from parallel lists using `Record.FromList`.
- Convert a Name/Value table to a record with `Record.FromTable`.
- Implement the defaults-and-overrides pattern using `Record.Combine`.

Syntax	<code>Record.TransformFields(record as record, transformOperations as list, optional missingField as nullable number) as record</code>	
Parameter	Type	Description
<code>record</code>	record	The source record.
<code>transformOperations</code>	list	A list of {fieldName, transformFunction} pairs. The function receives the current field value and returns the new value.
<code>missingField</code>	number	Optional. <code>MissingField.Ignore</code> or <code>MissingField.Error</code> (default).
Returns	Returns a new record with the specified fields replaced by the result of applying the corresponding transform functions. Fields not listed are carried over unchanged.	
Example	<code>Record.TransformFields([Name=" alice ", Score="95"], { {"Name", Text.Trim}, {"Score", Number.FromText} })</code>	
Result	<code>[Name="alice", Score=95]</code>	

NOTE

`Record.TransformFields` can only modify existing fields — it cannot add new ones. To both transform existing fields and add new ones, chain it with `Record.AddField`.

Syntax	Record.Combine(records as list) as record	
Parameter	Type	Description
records	list	A list of record values to merge in order.
Returns	Merges all records into a single record. When field names conflict, the last record wins. Fields present in only one record are included unchanged.	
Example	Record.Combine({[A=1, B=2], [B=99, C=3]})	
Result	[A=1, B=99, C=3]	

NOTE

The last-record-wins rule enables the defaults-and-overrides pattern: place the Defaults record first and the user Options record last. Fields the user does not supply fall back to the defaults. This is the idiomatic way to handle optional function parameters in M.

Defaults-and-overrides pattern with Record.Combine:

```
// Config-driven function with optional parameters
TransformSales = (
    source as table,
    optional options as nullable record
) as table =>
    let
        Defaults = [ RoundDecimals=2, RemoveNulls=true, MaxRows=null ],
        Opts = Record.Combine({ Defaults, options ?? [] }), // ?? [] handles null
        Rounded = Table.TransformColumns(source,
            {{"Amount", each Number.Round(_, Opts[RoundDecimals])}}),
        Filtered = if Opts[RemoveNulls]
            then Table.SelectRows(Rounded, each [Amount] <> null)
            else Rounded,
        Limited = if Opts[MaxRows] = null then Filtered
            else Table.FirstN(Filtered, Opts[MaxRows])
    in
        Limited

// Caller only specifies what they want to change:
TransformSales(FactSales, [ MaxRows = 1000 ]) // uses all other defaults
```

Syntax	<code>Record.FromList(list as list, fields as any) as record</code>	
Parameter	Type	Description
<code>list</code>	list	The values for each field.
<code>fields</code>	list or type	A list of field name strings, or a record type.
Returns	Constructs a record from a parallel list of values and a list of field names. The n-th value becomes the value of the n-th field.	
Example	<code>Record.FromList({10, "London", true}, {"Score","City","Active"})</code>	
Result	<code>[Score=10, City="London", Active=true]</code>	

NOTE

Pair with `Record.FieldValues` and `Record.FieldNames` to round-trip a record through a list. Also used with `List.Zip`: `Record.FromList(Values, Names)`.

Syntax	<code>Record.FromTable(table as table) as record</code>	
Parameter	Type	Description
<code>table</code>	table	A two-column table with columns named exactly <code>Name</code> (text) and <code>Value</code> (any). Each row becomes one field in the resulting record.
Returns	Converts a Name/Value table into a record. The inverse of <code>Record.ToTable</code> .	
Example	<code>Record.FromTable(#table({"Name","Value"},{{"Color","Blue"}, {"Size","L"}}))</code>	
Result	<code>[Color="Blue", Size="L"]</code>	

NOTE

This is the cornerstone of config-driven Power Query. Store connection strings, thresholds, and mappings in a SharePoint or Excel table with Name/Value columns, load it as a staging query, and call `Record.FromTable` to access all settings by name without hard-coding anything in M.

LESSON 5

Records in Tables — Row-Level Patterns

After completing this lesson, you will be able to:

- Explain that each `_` in table functions delivers the current row as a record.
- Add multiple computed columns in a single step using a record-valued column.
- Implement an inline lookup without `Table.Join` using `Record.FieldOrDefault`.

each `_` Is a Record

Inside `Table.AddColumn`, `Table.SelectRows`, `Table.TransformColumns`, and all similar table iteration functions, the implicit parameter `_` is the current row as a record. Every field in the table is accessible as a named field on that record:

```
// All three forms are equivalent – choose the most readable:
Table.AddColumn( Sales, "Tax", each _[Amount] * 0.1 )
Table.AddColumn( Sales, "Tax", each Record.Field(_, "Amount") * 0.1 )
Table.AddColumn( Sales, "Tax", (row) => row[Amount] * 0.1 )

// each is shorthand for (row) => expression
// _ IS the record – it has all table columns as fields
```

Multi-Column Computed Record Pattern

When multiple new columns all depend on the same source fields, add a single record-valued column (all computations in one expression), then expand it. This is cleaner, more readable, and evaluates source fields only once per row:

```
let
    WithCalcs = Table.AddColumn(
        Sales,
        "Metrics", // intermediate column name
        each [
            GrossMargin = [Revenue] - [Cost],
            MarginPct = ([Revenue] - [Cost]) / [Revenue],
            IsHighValue = ([Revenue] - [Cost]) / [Revenue] > 0.30,
            Label = [Region] & " | " & [Product]
        ],
        type record
    ),
    // Expand the record column into four separate columns
    Result = Table.ExpandRecordColumn(
        WithCalcs, "Metrics",
        { "GrossMargin", "MarginPct", "IsHighValue", "Label" }
    )
in
    Result
```



Naming the intermediate column descriptively (e.g. "Metrics", "Calculations") makes the query self-documenting. A colleague reading the query immediately understands that the expansion step unpacks a bundle of related computations — versus seeing five separate `Table.AddColumn` steps with no obvious connection.

Inline Lookup Without Table.Join

For small reference tables (under ~200 rows), converting the lookup to a record and using Record.FieldOrDefault per row is faster to write and avoids a full join:

```
let
// Step 1: Build a lookup record once (keyed on the code column)
RegionLookup = Record.FromList(
LookupTable[FullName],
LookupTable[Code]
),
// Step 2: Apply per row – safe with a fallback for unmatched codes
Result = Table.AddColumn(
Sales,
"RegionName",
each Record.FieldOrDefault( RegionLookup, [RegionCode], "Unknown" ),
type text
)
in
Result

// For larger lookups (thousands of rows), use Table.Join instead.
```

LESSON 6

Metadata and Type Operations

After completing this lesson, you will be able to:

- Attach arbitrary metadata to any M value using `Value.ReplaceMetadata`.
- Read metadata back with `Value.Metadata`.
- Annotate column types with metadata records to create self-describing datasets.
- Build a dynamic `TypeMap` record and use it to drive `Table.TransformColumnTypes`.

Syntax	<code>Value.Metadata(value as any) as record</code>	
Parameter	Type	Description
<code>value</code>	any	Any M value — table, list, record, or scalar.
Returns	Returns the metadata record attached to the value. Returns an empty record <code>[]</code> if no metadata has been attached.	
Example	<code>Value.Metadata(Value.ReplaceMetadata(42, [Source="CRM"]))</code>	
Result	<code>[Source="CRM"]</code>	

NOTE

Metadata survives most operations that preserve the value's type. It is stripped by operations that produce a new type (e.g. `Table.Sort` produces a new table, potentially without metadata). Store metadata in a variable before such operations.

Syntax	<code>Value.ReplaceMetadata(value as any, metadataRecord as record) as any</code>	
Parameter	Type	Description
<code>value</code>	any	The value to annotate.
<code>metadataRecord</code>	record	The metadata to attach. Replaces any existing metadata.
Returns	Returns the value with the specified metadata record attached. The value itself is unchanged.	
Example	<code>Value.ReplaceMetadata(SalesTable, [Author="Finance", Version=3])</code>	
Result	<code>SalesTable with metadata [Author="Finance", Version=3]</code>	

NOTE

Metadata is the correct mechanism for tracking provenance, refresh timestamps, source system identifiers, and quality flags without adding columns to the table. Read it back with `Value.Metadata(table)[Author]`.

Dynamic Type Builder for `Table.TransformColumnTypes`

Hard-coding a long list of {"Column", type value} pairs is fragile and tedious. Storing type assignments in a TypeMap record makes them trivial to maintain:

```
// TypeMap record: field name = target M type
TypeMap = [
    Date = type date,
    Revenue = type number,
    Quantity = Int64.Type,
    Country = type text,
    IsActive = type logical
],

// Build the pairs list dynamically – no hard-coding
TypePairs = List.Zip({
    Record.FieldNames(TypeMap),
    Record.FieldValues(TypeMap)
}),

// Apply
Result = Table.TransformColumnTypes(Source, TypePairs)

// To add or change a type: edit the TypeMap record only.
// The List.Zip step is reusable across all queries.
```

TIP

Store the TypeMap in a separate staging query named "Config_Types". Reference it from every query that needs the same column types. When a source changes a column type, you update one record in one query — all dependent queries pick up the change automatically.

LESSON 7

Advanced Record Patterns for PL-300

After completing this lesson, you will be able to:

- Build a config-driven query that reads all parameters from an Excel table.
- Compare adjacent rows using indexed table access and record field arithmetic.
- Flatten nested JSON records into a flat table without manual expand steps.
- Implement a parameterised column selector function using record field names.
- Apply the defaults-and-overrides function option pattern.

Pattern 1: Config-Driven Query with Record.FromTable

Hard-coding server names, database names, and filter thresholds in M is an anti-pattern that forces code changes for every environment migration. The correct approach stores all parameters in a configuration table:

```
// Config table in Excel: Name | Value
// ServerName | prod-sql-01
// DatabaseName | SalesDB
// MaxRows | 100000
// StartDate | 2024-01-01

let
    ConfigTable = Excel.Workbook(File.Contents("Config.xlsx"))
    {[Name="Params"]}[Data],
    Config = Record.FromTable(ConfigTable), // one-time conversion

    // Access parameters by name – no positional indexing
    Source = Sql.Database(Config[ServerName], Config[DatabaseName]),
    Sales = Source{[Name="Sales"]}[Data],
    Limited = Table.FirstN(Sales, Number.FromText(Config[MaxRows])),
    DateFrom = Date.From(Config[StartDate]),
    Result = Table.SelectRows(Limited, each [Date] >= DateFrom)
in
    Result
// Change parameters in Excel → zero M code changes needed
```

Pattern 2: Row Comparison (Previous Row Delta)

```
// Calculate change from previous row – requires index for row access
let
    Indexed = Table.AddIndexColumn(Sales, "Idx", 0, 1, Int64.Type),

    WithDiff = Table.AddColumn(
        Indexed, "ChangeFromPrev",
        each
            let
                prevRow = try Indexed{[Idx]-1} otherwise null
            in
                if prevRow = null then null
                else [Amount] - prevRow[Amount],
        type nullable number
    ),
    Result = Table.RemoveColumns(WithDiff, {"Idx"})
in
    Result
```

```
Result
// try ... otherwise null handles the first row safely
```

Pattern 3: Flattening Nested JSON Records

```
// API returns rows where one column contains a nested record
// e.g. { "id":1, "user":{"name":"Alice","email":"a@b.com"}, "amount":99 }

let
Raw = Json.Document(Web.Contents("https://api.example.com/orders")),
Tbl = Table.FromList(Raw, Splitter.SplitByNothing(), {"Row"}),
Expanded = Table.ExpandRecordColumn(Tbl, "Row", {"id","user","amount"}),

// Flatten nested [user] record – safe access with fallback
WithName = Table.AddColumn(Expanded, "UserName",
each Record.FieldOrDefault([user], "name", null), type text),
WithEmail = Table.AddColumn(WithName, "UserEmail",
each Record.FieldOrDefault([user], "email", null), type text),
Result = Table.RemoveColumns(WithEmail, {"user"})
in
Result
```

Pattern 4: Parameterised Column Selector

```
// A reusable function: specify which columns to keep via a record
// The record's field names are the columns to select
SelectByRecord = (tbl as table, colSpec as record) as table =>
let
Wanted = Record.FieldNames(colSpec),
Present = List.Intersect({Wanted, Table.ColumnNames(tbl)}),
Missing = List.Difference(Wanted, Present)
in
if List.IsEmpty(Missing)
then Table.SelectColumns(tbl, Wanted)
else error "Missing columns: " & Text.Combine(Missing, ", ")

// Call it – the record's field names define which columns to keep
Result = SelectByRecord(
CustomersTable,
[ CustomerID = 1, Name = 1, Email = 1 ]
)
// Values are ignored – only field names matter
```

NOTE

Pattern 4 uses a record as a named-set of column identifiers — the values are irrelevant, only the field names matter. This technique appears in PL-300 exam scenarios involving dynamic schema selection and reusable transformation functions.

DEMONSTRATION 1

Config Record from an Excel Parameter Table

In this demonstration the instructor builds a fully config-driven query that reads all connection parameters from an Excel table, converts them to a record with `Record.FromTable`, and uses the record to drive a SQL query — with zero hard-coded values in M.

Scenario: Contoso's Power BI model must connect to different SQL Server databases in Development, UAT, and Production. Changing environments currently requires editing M code. The goal is to eliminate that requirement.

Demonstration Steps

1	Open the companion file <code>Config_Parameters.xlsx</code> . Inspect the Params sheet — it has columns: Name and Value, with rows for <code>ServerName</code> , <code>DatabaseName</code> , <code>MaxRows</code> , and <code>StartDate</code> .
2	In Power BI Desktop, load <code>Config_Parameters.xlsx</code> as a staging query named <code>Config_Raw</code> . Do not load it to the model (right-click > Enable Load: off).
3	Create a new blank query named <code>Config</code> . Add: <code>Config = Record.FromTable(Config_Raw)</code> . Set 'in Config' and observe the record in the Query Preview pane.
4	Create a new blank query named <code>FactSalesDB</code> . Add the <code>Sql.Database</code> call using <code>Config[ServerName]</code> and <code>Config[DatabaseName]</code> instead of literal strings.
5	Add a <code>MaxRows</code> filter step: <code>Table.FirstN(Source, Number.FromText(Config[MaxRows]))</code> .
6	Change the <code>ServerName</code> value in <code>Config_Parameters.xlsx</code> . Refresh the model. Observe that the SQL connection automatically targets the new server — no M edits required.
7	Discuss: why this pattern is superior to Power BI parameters for multi-environment deployments. (Config tables can be on SharePoint; parameters require Desktop edits.)

RESULT

A config-driven query that reads all connection parameters from Excel. Changing the server, database, or row limit requires only an Excel edit — no M code changes. The same M query works across Development, UAT, and Production.

DEMONSTRATION 2

Flattening a Nested JSON API Response

In this demonstration the instructor consumes a REST API that returns nested JSON objects, shows how records emerge naturally from `Json.Document`, and uses `Table.ExpandRecordColumn`, `Record.HasFields`, and `Record.FieldOrDefault` together to flatten the response into a clean, typed table.

Scenario: A product API returns order records where the customer details are embedded as a nested object. The nesting depth and field presence varies between records (some orders have a shipping address; others do not).

Demonstration Steps

1	Load the sample JSON file (or fetch from the demo API). Show <code>Json.Document</code> output in the Query Preview — observe that nested objects appear as <code>[Record]</code> values.
2	Add <code>Table.FromList</code> with <code>Splitter.SplitByNothing()</code> to convert the list to a table with one <code>[Record]</code> column per row.
3	Use <code>Table.ExpandRecordColumn</code> to expand the top-level fields. Show that nested customer records still appear as <code>[Record]</code> values in the expanded table.
4	Add <code>Record.HasFields</code> inside <code>Table.AddColumn</code> to check whether the customer record has a shipping address before accessing it — preventing errors on incomplete records.
5	Use <code>Record.FieldOrDefault</code> to extract specific nested fields with null fallbacks: <code>Record.FieldOrDefault([customer], "shippingCity", "(no city)")</code> .
6	Apply the dynamic <code>TypeMap</code> builder from Lesson 6 to type all resulting columns in one step, using a <code>TypeMap</code> record defined at the top of the query.
7	Demonstrate adding <code>Value.ReplaceMetadata</code> to attach a refresh timestamp and source URL to the final table for lineage tracking.

RESULT

A flat, fully typed table built from a deeply nested JSON API response. All nested records are safely flattened using `Record.FieldOrDefault` — no errors on records with missing optional fields. Metadata captures the source URL and refresh time for lineage without adding columns to the table.

LAB 5

Record Functions

In this lab you apply the full range of Record.* functions — from basic field access and inspection through modification, row-level table patterns, and config-driven dynamic type schemas. Work with the FactSales, DimProduct, and Config_Raw queries from previous modules and the companion demonstration materials.

Lab Scenario

Contoso Analytics needs several enhancements to their Power BI model. You must: inspect and validate record structures from the API, clean and normalise customer records using modification functions, add multiple computed columns efficiently using the record-expand pattern, and build a config-driven type transformation function that requires zero code changes to adapt to new source schemas.

NOTE

All exercises use M queries only — no ribbon clicks. Write all code in the Advanced Editor. Label every step clearly. Exercises build on each other — complete them in order.

Exercise 1: Record Access and Inspection (Estimated time: 20 minutes)

In this exercise you inspect a record's structure and safely access its fields.

■ Create a blank query named RecordInspection. In the Advanced Editor write a `let . . . in` expression with the following named steps:

- `Customer = [ CustomerID=1042, Name=" Northwind Traders ", Email="contact@northwind.com", Region="EU", CreditLimit=50000, Active=true, Tags={"premium", "b2b"} ]`
- `FieldList = Record.FieldNames(Customer)` — list all field names.
- `FieldCount = Record.FieldCount(Customer)` — how many fields?
- `HasPhone = Record.HasFields(Customer, {"Phone", "Email"})` — does Phone exist?
- `SafePhone = Record.FieldOrDefault(Customer, "Phone", "N/A")` — safe access.
- `CleanName = Text.Trim(Record.Field(Customer, "Name"))` — use `Record.Field`, not `[]`.
- `AsTable = Record.ToTable(Customer)` — convert to Name/Value table.
- Cycle through each step as the `in` clause. Verify: `FieldCount=7`, `HasPhone=false`, `SafePhone="N/A"`, `CleanName="Northwind Traders"`.

RESULT

RecordInspection returns: `FieldList={"CustomerID", "Name", ..., "Tags"}`, `FieldCount=7`, `HasPhone=false`, `SafePhone="N/A"`, `CleanName="Northwind Traders"`, `AsTable` is a 7-row Name/Value table.

Exercise 2: Creating and Modifying Records (Estimated time: 20 minutes)

In this exercise you clean, normalise, and enrich the Customer record using the full suite of modification functions.

- Starting from the Customer record in Exercise 1, add a `LastUpdated` field using `DateTime.LocalNow()`.
- Remove the `Tags` field — it is not supported by the downstream schema.
- Rename `CustomerID` to `ID` and `CreditLimit` to `CreditLimitUSD`.
- Transform the `Name` field to trimmed title case and `Region` to upper case using `Record.TransformFields`.

- Combine with a Defaults record [Source="CRM", Version=2, QA_Passed=false] using Record.Combine. Place Defaults first so the customer record overrides.
- STRETCH: Use Record.SelectFields to extract only ID, Name, and Email into a new record.

```
// Task 4 hint – TransformFields
Record.TransformFields(
Customer,
{
{ "Name", (v) => Text.Proper(Text.Trim(v)) },
{ "Region", Text.Upper }
}
)

// Task 5 hint – Defaults first, customer overrides
Defaults = [ Source="CRM", Version=2, QA_Passed=false ],
Record.Combine({ Defaults, CleanedCustomer })
```

RESULT

The final record has: ID=1042, Name="Northwind Traders", Email="contact@northwind.com", Region="EU", CreditLimitUSD=50000, Active=true, LastUpdated=(current datetime), Source="CRM", Version=2, QA_Passed=false.

Exercise 3: Records in Tables — Row-Level Patterns (Estimated time: 25 minutes)

In this exercise you apply the row-level record patterns to FactSales — adding multiple computed columns in one step and performing an inline lookup without Table.Join.

- Load FactSales (columns: OrderID, Region, Product, Quantity, UnitPrice).
- Build a lookup record from DimRegion (columns: Code, FullName): RegionLookup = Record.FromList(DimRegion[FullName], DimRegion[Code]).
- Add a Revenue column: each [Quantity] * [UnitPrice].
- Add a RegionName column using Record.FieldOrDefault on RegionLookup per row.
- Add a record-valued Metrics column containing GrossRevenue, IsHighValue (Revenue > 5000), and Label (Region & "-" & Product).
- Use Table.ExpandRecordColumn to expand Metrics into three separate columns.
- STRETCH: Add a previous-row delta column using Table.AddIndexColumn and try Indexed{[Idx]-1} otherwise null.

RESULT

FactSales has new columns: Revenue (number), RegionName (text with fallback "Unknown" for unmatched codes), GrossRevenue (=Revenue), IsHighValue (logical), Label (text). All from a single pass through the table.

Exercise 4: Config Records and Dynamic Type Schemas (Estimated time: 25 minutes)

In this exercise you build a reusable, config-driven transformation function that accepts a TypeMap record and a ColumnMap record, applying both to standardise an incoming raw table. This is the enterprise pattern for handling multiple similarly-structured source files with different column names and types.

- Load Config_Parameters.xlsx as Config_Raw. Apply Record.FromTable to produce a Config record. Access Config[StartDate] and Config[MaxRows] in a FactSales filter step.

- Create a blank query named TransformTable. Define a function with signature (rawTable as table, TypeMap as record, ColumnMap as record) as table.
- Inside the function: (a) build rename pairs from ColumnMap using List.Zip({Record.FieldNames(ColumnMap), Record.FieldValues(ColumnMap)}), (b) apply Table.RenameColumns, (c) build type pairs from TypeMap using the same List.Zip pattern, (d) apply Table.TransformColumnTypes.
- Add schema validation: use Record.HasFields or List.Difference to detect any TypeMap columns still missing after rename, and raise a descriptive error.
- Call the function with a sample TypeMap [Date=type date, Revenue=type number, Region=type text] and a ColumnMap [sale_dt="Date", rev_usd="Revenue", rgn_cd="Region"] applied to a raw CSV table.

```
// Full solution skeleton
TransformTable = (rawTable as table, TypeMap as record, ColumnMap as record) as table =>
let
    OldNames = Record.FieldNames(ColumnMap),
    NewNames = Record.FieldValues(ColumnMap),
    RenamePairs = List.Zip({OldNames, NewNames}),
    Renamed = Table.RenameColumns(rawTable, RenamePairs, MissingField.Ignore),

    TypeNames = Record.FieldNames(TypeMap),
    TypeValues = Record.FieldValues(TypeMap),
    TypePairs = List.Zip({TypeNames, TypeValues}),
    Typed = Table.TransformColumnTypes(Renamed, TypePairs),

    Missing = List.Difference(TypeNames, Table.ColumnNames(Typed))
in
    if List.IsEmpty(Missing) then Typed
    else error "Missing columns: " & Text.Combine(Missing, ", ")
```

RESULT

TransformTable correctly renames and types columns from any raw source using only the two config records. Adding or removing a column requires only editing the records — no changes to the function body. The schema validation step raises a clear, actionable error if a renamed column is still missing after the rename step.

Module Review and Takeaways

Review Questions

What is the difference between `rec[Field]` and `Record.Field(rec, "Field")`?

Answer: `rec[Field]` is resolved at compile time — the field name must be a literal string known at design time. `Record.Field()` accepts any text expression as the field name, making it the only option for runtime-dynamic field access. Use `Record.Field` when the field name is stored in a variable or computed from a list.

Which function safely returns a default value when a field is absent from a record?

Answer: `Record.FieldOrDefault(record, fieldName, defaultValue)`. It returns `defaultValue` instead of throwing an error when the field does not exist. This is the correct choice for all lookup operations where unmatched keys are possible.

How does `Record.Combine` handle field name conflicts, and what pattern does this enable?

Answer: The last record in the list wins. This enables the defaults-and-overrides pattern: place the Defaults record first and the user Options record last. Fields the caller does not supply fall back to the defaults automatically.

How do you convert a two-column Name/Value table into a record, and why is this useful?

Answer: `Record.FromTable(table)`. The table must have columns named exactly `Name` and `Value`. This is the cornerstone of config-driven Power Query: store all parameters in a SharePoint or Excel table, load it as a staging query, and call `Record.FromTable` to access all settings by name without hard-coding anything in M.

What is the advantage of adding a record-valued column and then expanding it, versus adding multiple separate `Table.AddColumn` steps?

Answer: The record-valued column approach evaluates all source fields once per row and groups all related computations in a single, readable expression. Multiple separate `Table.AddColumn` steps may re-evaluate the same source fields repeatedly and are harder to read when the computations are logically related.

In a `Table.AddColumn` expression, what type is the `_` parameter?

Answer: A record representing the current row. Every column in the table is accessible as a named field on that record, using either `rec[FieldName]` or `Record.Field(_, "FieldName")`. The `each` keyword's `_` is always the current row record in table iteration functions.

How do you build the argument list for `Table.TransformColumnTypes` dynamically from a `TypeMap` record?

Answer: Combine `Record.FieldNames(TypeMap)` and `Record.FieldValues(TypeMap)` with `List.Zip`: `List.Zip({Record.FieldNames(TypeMap), Record.FieldValues(TypeMap)})`. This produces the required list of {columnName, type} pairs without hard-coding. Adding or removing a column type requires only editing the `TypeMap` record.

What does `Record.TransformFields` do that `Record.AddField` cannot?

Answer: `Record.TransformFields` modifies the values of existing fields by applying a function to each. `Record.AddField` can only add new fields — it cannot change the value of an existing field. To both transform existing fields and add new ones, chain both functions.

Best Practices

- Always use `Record.FieldOrDefault` for lookup operations where the key may not exist. `Record.Field` throws on missing fields; `Record.FieldOrDefault` returns your chosen default.
- Use `Record.Field()` rather than `rec[Field]` whenever the field name is computed at runtime. The `[]` syntax requires a compile-time literal and cannot accept a variable.
- Apply the defaults-and-overrides pattern with `Record.Combine` for all optional function parameters. Place Defaults first; the caller's record last. This is cleaner and more extensible than chains of if-then-else null checks.
- Validate schema before transformation using `Record.HasFields`. In enterprise ETL, always confirm required fields are present before accessing them. Provide a descriptive error message so the failure reason is immediately clear.
- Store connection strings, thresholds, and lookup mappings in a Name/Value config table, not as literals in M. Convert with `Record.FromTable`. Access by name. Never hard-code.
- Use `Record.TransformFields` for bulk type coercion of multiple fields. It is cleaner and more maintainable than chained `Table.TransformColumnTypes` calls on a per-field basis.
- Build `TypeMap` records for `Table.TransformColumnTypes`. A `TypeMap` record with `{ColumnName: type}` pairs, combined with `List.Zip`, produces the transformation list dynamically — one place to edit.
- Always provide explicit field name lists to `Table.ExpandRecordColumn`. Avoid the auto-detect option, which can break if the source schema changes between refreshes.
- Treat the `_` record in each expressions as read-only. Produce modified versions with `Record.AddField` or `Record.TransformFields`; do not attempt in-place mutation.
- Use meaningful names for intermediate record columns (e.g. "Metrics", "Calculations") before expanding them. This makes the query self-documenting for collaborators.

Common Issues and Solutions

Expression.Error: The field 'X' of the record wasn't found

Cause: You accessed a field that does not exist in the record. Causes: typo in field name, schema change in source, or a dynamic key that is not present in this row.

Solution: Use `Record.HasFields` before accessing, or switch to `Record.FieldOrDefault` with a fallback. During development, set `'in Record.FieldNames(_)` to inspect what fields are available.

Cannot use a variable as a field name in `[]` syntax

Cause: You wrote `rec[myVar]` expecting the variable to be evaluated as a field name — this fails because `[]` requires a compile-time literal string.

Solution: Replace `rec[myVar]` with `Record.Field(rec, myVar)`. This is the only correct form for runtime-dynamic field names.

Record.Combine produces unexpected field values

Cause: The wrong record is winning on conflicts — usually the Defaults record is listed last instead of first, so it overrides the user options rather than the other way around.

Solution: Remember: last record wins. Always list defaults first: `Record.Combine({Defaults, UserOptions})`.

Record.FromTable fails with a 'column not found' error

Cause: The input table does not have columns named exactly Name and Value — they may be capitalised differently or named something else (e.g. Key / Setting).

Solution: Use Table.RenameColumns to normalise the column names to Name and Value before passing to Record.FromTable. Check the source table column names first.

Metadata is lost after certain transformation steps

Cause: Operations including Table.Sort, Table.Group, and Table.Join may strip the metadata record from the result — the new table is a fresh value without the original annotation.

Solution: Store the metadata record in a variable before the operation and re-attach it afterwards: Value.ReplaceMetadata(TransformedTable, originalMeta).

Key Functions — Module 5 Quick Reference

All record functions introduced in this module, organised by category.

Function	Category	Key Behaviour
<code>Record.Field(r, f)</code>	Access	Return value of field <i>f</i> ; field name may be a variable
<code>Record.FieldOrDefault(r, f, d)</code>	Access	Value of field <i>f</i> , or default <i>d</i> if field is absent
<code>Record.FieldNames(r)</code>	Inspect	List of all field names in field order
<code>Record.FieldValues(r)</code>	Inspect	List of all field values in field order
<code>Record.FieldCount(r)</code>	Inspect	Number of fields in the record
<code>Record.HasFields(r, f)</code>	Inspect	true if ALL named field(s) exist in the record
<code>Record.ToTable(r)</code>	Convert	Two-column Name/Value table from a record
<code>Record.ToList(r)</code>	Convert	List of field values (alias for FieldValues)
<code>Record.FromList(vals, names)</code>	Create	Build record from parallel values and names lists
<code>Record.FromTable(tbl)</code>	Create	Convert Name/Value table back to a record
<code>Record.AddField(r, f, v)</code>	Modify	New record with additional field <i>f</i> = <i>v</i>
<code>Record.RemoveFields(r, f)</code>	Modify	New record with field(s) removed
<code>Record.RenameFields(r, pairs)</code>	Modify	New record with fields renamed per {old,new} pairs
<code>Record.SelectFields(r, f)</code>	Modify	New record with only specified fields retained
<code>Record.TransformFields(r, ops)</code>	Transform	Apply functions to existing fields; return new record
<code>Record.Combine({r1, r2...})</code>	Combine	Merge list of records; last record wins on conflicts
<code>Value.Metadata(v)</code>	Metadata	Return the metadata record attached to value <i>v</i>
<code>Value.ReplaceMetadata(v, m)</code>	Metadata	Attach metadata record <i>m</i> to value <i>v</i>
<code>Value.ReplaceType(v, t)</code>	Type	Ascribe type <i>t</i> to value <i>v</i> without data conversion
<code>Value.Type(v)</code>	Type	Return the M type of value <i>v</i>
<code>Type.Is(t1, t2)</code>	Type	Check if type <i>t1</i> is compatible with type <i>t2</i>

Module Summary

- A record is an ordered collection of named field-value pairs enclosed in square brackets. Every table row is a record; every each `_` expression in a table function receives the current row as a record.
- Field access has two forms: `rec[Field]` (compile-time literal only) and `Record.Field(rec, varName)` (runtime-dynamic). Always use `Record.Field` when the field name is computed. Use `Record.FieldOrDefault` for safe access with a fallback.
- Records are immutable. `Record.AddField`, `RemoveFields`, `RenameFields`, `SelectFields`, and `TransformFields` all return new records — the original is unchanged. This makes transformations composable and side-effect-free.
- `Record.Combine` merges a list of records; the last record wins on conflicts. This enables the defaults-and-overrides pattern for optional function parameters — the idiomatic M alternative to long chains of null checks.
- `Record.FromTable` is the key to config-driven queries: store all parameters in a Name/Value table on SharePoint or Excel, load it, call `Record.FromTable`, and access settings by name. No M code changes are needed to switch environments.
- The multi-column record-expand pattern — adding a single record-valued column then expanding it — is cleaner and more readable than multiple separate `Table.AddColumn` steps when all computations share the same source fields.
- The dynamic TypeMap builder (`Record.FieldNames + Record.FieldValues + List.Zip`) eliminates hard-coded `Table.TransformColumnTypes` lists. All type changes require editing only the TypeMap record.
- Five professional patterns: config-driven queries, row comparison, nested JSON flattening, parameterised column selection, and defaults-and-overrides — all driven by record functions.

NOTE

Next Module — Module 6: Table Functions — Core. Covers all core `Table.*` functions: filtering, grouping, joining, pivoting, unpivoting, and the advanced patterns that underpin every real-world Power BI data model. Module 6 has the heaviest coverage in the PL-300 exam.